

RECON ROVER V2

OFFICIAL TECHNICAL DESIGN & FEATURE MANUAL

Version: 1.0.0 Architecture Version: 8.9 Document Version: 1.0 Project Repository:
Recon Rover V2

Revision History

Date	Version	Description	Author
2026-07-16	1.0	Initial Release of Phase 8.9 Technical Manual	AI Engineering Team

2. Executive Summary

Recon Rover V2 is an advanced, autonomous robotic platform designed to perform complex reconnaissance, reasoning, and environmental interaction without human intervention. Built upon a federated architecture separating low-level hardware control (ESP32) from high-level cognitive processing (Raspberry Pi), the rover integrates cutting-edge Artificial Intelligence paradigms including Retrieval-Augmented Generation (RAG), multi-agent consensus networks, real-time computer vision, and dynamic resource optimization. This manual serves as the definitive engineering reference for the system's software architecture, AI runtimes, hardware bridging, and operational flow.

3. Table of Contents

- 1. Cover Page
- 2. Executive Summary
- 3. Table of Contents
- 4. Project Overview
- 5. Complete Feature List
- 6. Complete Hardware Architecture
- 7. Complete Software Architecture
- 8. AI Architecture
- 9. Runtime Architecture
- 10. Communication Architecture
- 11. EventBus Documentation
- 12. Data Flow
- 13. Control Flow
- 14. Runtime Flow
- 15. Performance Specifications
- 16. Safety Architecture
- 17. Engineering Decisions
- 18. Known Limitations
- 19. Future Expansion
- 20. Glossary
- 21. References

4. Project Overview

Mission

To provide a fully autonomous, highly intelligent reconnaissance platform capable of real-time environmental analysis, semantic mapping, and multi-agent reasoning in unstructured environments.

Design Philosophy

The system follows a strict decoupling between physical actuation and cognitive reasoning. High-frequency safety constraints and motor controls are delegated to the microcontroller (ESP32), while all AI, vision, and planning modules operate on a high-throughput microcomputer (Raspberry Pi). All inter-process communication occurs asynchronously over a centralized EventBus.

Capabilities & Features

- Autonomous spatial navigation and obstacle avoidance.
- Real-time semantic understanding of visual data.
- Conversational command processing and speech synthesis.
- Multi-agent collaboration for complex decision making.
- Continuous system benchmarking and resource optimization.

Target Applications

- Hazardous environment exploration.
- Automated security patrolling and anomaly detection.
- Search and rescue support.
- Academic AI and robotics research.

5. Complete Feature List

Hardware Features

- **ESP32 Core:** Handles PWM motor control, ultrasonic sensor arrays, and battery monitoring.
- **Raspberry Pi Core:** Serves as the AI brain, running async Python processes and model inference.
- **Sensors:** LiDAR (mapping), PiCamera (vision), Ultrasonic (proximity), IMU (orientation), Microphone (audio).

Software & AI Features

- **EventBus Integration:** System-wide message passing broker.
- **Vision Features:** Object detection, classification, depth estimation, and semantic segmentation.
- **Speech Features:** Wake-word detection, ASR (Speech-to-Text), TTS (Text-to-Speech), RTF profiling.
- **LLM Features:** Provider-agnostic inference (Ollama, vLLM, OpenAI), contextual

streaming, auto-failover.

- **RAG Features:** Vector database retrieval (ChromaDB/FAISS), hybrid search, context injection.
 - **Tool Calling:** Secure sandbox for LLMs to execute motor functions, query memory, or read diagnostics.
 - **Multi-Agent:** Blackboard context sharing, planner agents, vision agents, and consensus managers.
 - **Optimization:** Dynamic resource allocators, priority scheduling, thermal management.
 - **Benchmarking:** Passive telemetry collection spanning 13 different AI/Hardware domains.
-

6. Complete Hardware Architecture

Raspberry Pi (High-Level Brain)

- **Purpose:** Executes the AI Runtime, handles heavy computation, computer vision, and network bridging.
- **Operating System:** Linux (Raspberry Pi OS).
- **Interfaces:** I2C, SPI, USB (Camera, Audio), UART (ESP32 Serial connection).
- **Power:** 5V 3A via dedicated UBEC.

ESP32 (Low-Level Controller)

- **Purpose:** Hard-real-time control loop for motor drivers, encoders, and safety sensors.
- **Operating System:** FreeRTOS (C++).
- **Interfaces:** PWM (Motors), GPIO (Ultrasonic, Encoders), UART (Pi connection).
- **Power:** 3.3V logic, powered via voltage regulator from the main distribution board.

Interconnect

- **Serial (UART):** Configured at 115200 baud. The `SerialRuntime` on the Pi communicates with the ESP32 via structured JSON/Protocol Buffer packets.
-

7. Complete Software Architecture

The software stack evolved over 8 major phases. The `MAIN_CODE/RASPBERRY_PI/core/` directory is split into domains.

Phase 1: Core Framework

Established the basic async event loop and the `EventBus`, allowing decoupled publish-subscribe behavior.

Phase 2: Sensor & Motor Runtimes

Implemented `UltrasonicRuntime` and `MotorRuntime` to interface over serial with the ESP32 firmware, abstracting physical hardware into async event streams.

Phase 3 & 4: Navigation and Mapping

Introduced `NavigationRuntime` and `MappingRuntime` for dead-reckoning, obstacle avoidance algorithms, and spatial coordinate tracking.

Phase 5 & 6: Communication and Security

Built `NetworkRuntime` and `SecurityRuntime` to handle external WebSocket connections from the Ground Station and encrypt telemetry.

Phase 7: UI & Dashboard

Provided a web-based interface for remote monitoring and teleoperation.

Phase 8.0 - 8.9: The AI Architecture

The most complex module, housing the cognitive stack:

- **8.1 Vision:** Frame capture, model inference, bounding box generation.
 - **8.2 Speech:** Audio streaming, VAD, ASR, TTS.
 - **8.3 LLM:** Prompt parsing, inference scheduling, generic provider abstraction.
 - **8.4 RAG:** Vectorization, chunking, retrieval ranking.
 - **8.5 Tools:** Execution sandbox, security validation, permission models.
 - **8.6 Agents:** Supervisor routing, worker execution, conflict resolution.
 - **8.7 Optimization:** Resource throttling, cache management, thermal backoff.
 - **8.8 Benchmark:** Passive telemetry, latency tracking, JSON export.
 - **8.9 Demo:** End-to-end integration orchestrator (`DemoRuntime`).
-

8. AI Architecture

The AI stack operates as a deeply integrated pipeline:

1. **Sensory Input:** `VisionRuntime` and `SpeechRuntime` continuously digitize the environment and broadcast perceptual events to the `EventBus`.
 2. **Memory Augmentation:** The `RAGRuntime` intercepts reasoning queries, searches the `VectorDatabase` for historical context, and injects it into the prompt.
 3. **Reasoning:** The `LlmRuntime` evaluates the prompt. If the prompt is complex, the `AgentRuntime` splits it into sub-tasks (e.g., Vision Agent + Navigation Agent) and uses the `ConsensusManager` to merge results.
 4. **Action Execution:** If the LLM determines an action is necessary, it emits a structured request to the `ToolRuntime`. The `ToolValidator` ensures the request is safe, executes the physical command (e.g., `move_forward`), and returns the physical outcome to the LLM.
 5. **Observation:** The `BenchmarkRuntime` continuously profiles this entire cycle without blocking execution, while the `OptimizationRuntime` dynamically adjusts thread counts based on CPU thermals.
-

9. Runtime Architecture

The system follows a strict, idempotent lifecycle:

- **Startup:** Initialization of `EventBus`, database connections, and hardware interfaces.

- **SystemReadiness:** A gating mechanism that polls every subsystem. The rover will not begin autonomous operations until all modules return a healthy status.
 - **Runtime:** The main async execution loop. All modules idle until triggered by EventBus messages.
 - **Recovery:** In the event of a subsystem crash, the `RecoveryManager` attempts to re-instantiate the failing module without halting the entire rover.
 - **Shutdown:** Triggered via EventBus. Safely flushes in-memory databases, terminates active agents, and stops all motors before process exit.
-

10. Communication Architecture

All inter-process and inter-device communication is standardized.

- **ESP32 <-> Pi:** Serial UART transferring serialized JSON packets (e.g., `{"cmd": "motor", "val": [255, 255]}`).
 - **Pi <-> Modules:** In-memory `asyncio` EventBus. Subsystems publish `dataclass` payloads to string-based topics.
 - **Pi <-> Ground Station:** WebSockets wrapping EventBus topics, enabling remote dashboards to mirror the rover's internal state in real-time.
-

11. EventBus Documentation

The EventBus is the central nervous system.

- **Publishers:** Sensors, AI Runtimes, System Managers.
 - **Subscribers:** AI Agents, Data Loggers, Navigation Controllers.
 - **Key Topics:**
 - `hardware.telemetry`: Motor speeds, battery voltage.
 - `vision.detection`: Output of object recognition.
 - `llm.response`: The stream of tokens from the reasoning engine.
 - `benchmark.telemetry`: Aggregated performance metrics.
-

12. Data Flow

1. Camera -> Pi Camera Driver -> VisionRuntime -> Frame Buffer -> ML Model -> Bounding Boxes -> EventBus -> Memory Agent -> RAG Database.
-

13. Control Flow

1. Microphone -> SpeechRuntime -> Text Transcript -> Agent Supervisor -> Planner Agent -> ToolRuntime -> EventBus -> SerialRuntime -> ESP32 -> Motor Drivers.
-

14. Performance Specifications

- **Concurrency:** Fully asynchronous. Network I/O and motor commands do not block

vision inference.

- **Memory Bound:** Vector databases and metric stores utilize bounded queues (`collections.deque`) to prevent out-of-memory (OOM) errors during continuous operation.
 - **Thermal Safety:** The `ThermalManager` actively scales back intensive LLM/Vision processing if the Pi SoC exceeds 75°C.
-

15. Safety Architecture

- **Hardware Layer:** ESP32 autonomously halts motors if ultrasonic sensors detect an immediate collision, regardless of Pi commands.
 - **Software Layer:** `ToolValidator` enforces strict permission schemas (e.g., an LLM cannot request to spin a motor at 5000 RPM).
-

16. Engineering Decisions

- **Why Python + AsyncIO?** Python offers the best ML ecosystem (PyTorch, LangChain, HuggingFace). AsyncIO allows the Pi to handle hundreds of concurrent I/O operations (sensors, network) while offloading heavy ML inference to background C++ threads or external APIs.
 - **Why EventBus?** Tight coupling leads to spaghetti code. The EventBus allows the Vision module to crash without taking down the Navigation module.
 - **Why Provider Agnostic LLMs?** The AI landscape changes daily. Abstracting the provider means swapping from OpenAI to a local Llama.cpp model requires changing a single config variable, not rewriting the application logic.
-

17. Known Limitations

- The system heavily relies on network latency if using cloud LLMs (OpenAI).
 - Raspberry Pi RAM (typically 4GB-8GB) limits the size of locally hosted LLMs.
-

18. Future Expansion

- **Phase 8.10+ (Deployment):** Over-the-air (OTA) updates, Docker containerization, and fleet management for multiple rovers operating in a swarm.
 - **LiDAR SLAM:** Integration of 2D/3D LiDAR for highly accurate metric mapping to complement semantic mapping.
-

19. Glossary

- **RAG:** Retrieval-Augmented Generation.
 - **LLM:** Large Language Model.
 - **EventBus:** A publish/subscribe messaging system.
 - **ESP32:** A low-cost, low-power system on a chip microcontroller.
-

20. References

- Recon Rover V2 Engineering Changelog
- Phase 8.0 - 8.9 Implementation Plans and Verification Reports

(End of Manual)